同期仕様書

Copyright (C) 2011-2014 by Center for Embedded Computing Systems
Graduate School of Information Science, Nagoya Univ., JAPAN
Copyright (C) 2011-2014 by FUJI SOFT INCORPORATED, JAPAN
Copyright (C) 2011-2013 by Spansion LLC, USA
Copyright (C) 2011-2013 by NEC Communication Systems, Ltd., JAPAN
Copyright (C) 2011-2014 by Panasonic Advanced Technology Development Co., Ltd., JAPAN
Copyright (C) 2011-2014 by Renesas Electronics Corporation, JAPAN
Copyright (C) 2011-2014 by Sunny Giken Inc., JAPAN
Copyright (C) 2011-2014 by TOSHIBA CORPORATION, JAPAN
Copyright (C) 2011-2014 by Witz Corporation, JAPAN

本ドキュメントは，次世代車載システム向けRTOSの仕様検討及び
開発に関するコンソーシアム型共同研究において作成したものである．
本ドキュメントの著作権は，上記の著作権者に帰属する．

上記著作権者は，以下の (1)?(3)の条件を満たす場合に限り，本ドキュメント（本ドキュメントを改変したものを含む．以下同じ）を使用・複製・改変・再配布（以下，利用と呼ぶ）することを無償で許諾する．
(1) 本ドキュメントを利用する場合には，上記の著作権表示，この利用条件および下記の無保証規定が，そのままの形でドキュメント中に含まれていること．
(2) 本ドキュメントを改変する場合には，ドキュメントを改変した旨の記述を，改変後のドキュメント中に含めること．ただし，改変後のドキュメントが，TOPPERSプロジェクト指定の開発成果物である場合には，この限りではない．
(3) 本ドキュメントの利用により直接的または間接的に生じるいかなる損害からも，上記著作権者およびTOPPERSプロジェクトを免責すること．また，本ドキュメントのユーザまたはエンドユーザからのいかなる理由に基づく請求からも，上記著作権者およびTOPPERSプロジェクトを免責すること．

本ドキュメントは，AUTOSAR（AUTomotive Open System ARchitecture）仕様に基づいている．上記の許諾は，AUTOSARの知的財産権を許諾するものではない．AUTOSARは，AUTOSAR仕様に基づいたソフトウェアを商用目的で利用する者に対して，AUTOSARパートナーになることを求めている．

本ドキュメントは，無保証で提供されているものである．上記著作権者およびTOPPERSプロジェクトは，本ドキュメントに関して，特定の使用目的に対する適合性も含めて，いかなる保証も行わない．また，本ドキュメントの利用により直接的または間接的に生じたいかなる損害に関しても，その責任を負わない．

用語定義と図の見方

- **MASTER_CORE**
  - –テスト管理用のメインタスクが存在するコア（マスタコア）
  - –図では赤色で示す
- **OTHER_CORE**
  - –MASTER_COREではないコア（他コア）
  - –図では青色で示す

図中の黒矢印の箇所を勝手に進ませないようにするために同期が用いられる

同期メカニズム

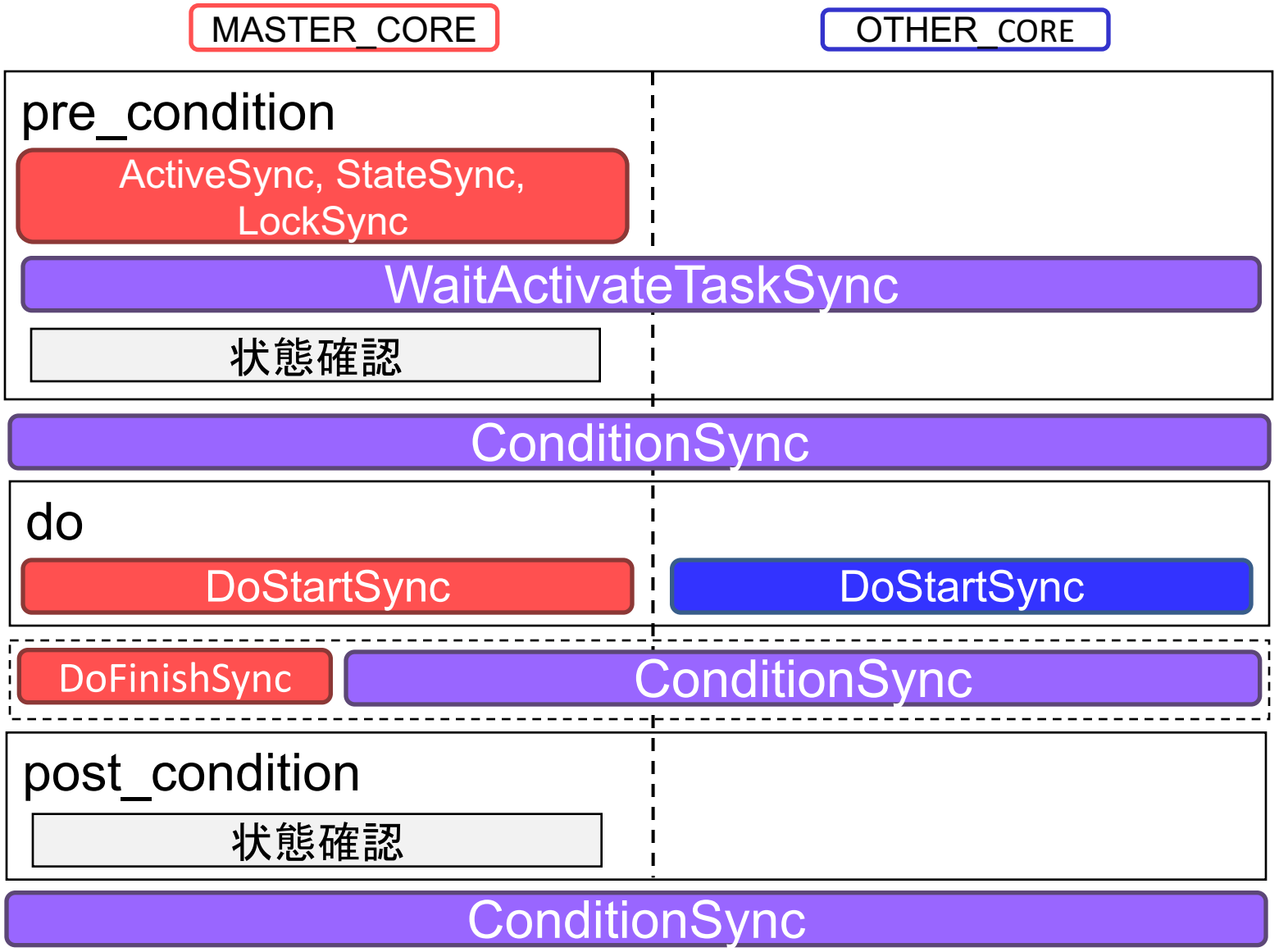
- **AktspBarrierSync**
  - 複数の指定したコア上にある実行状態の処理単位が，指定した処理を完了するまで互いに待機する
- **AktspMcWaitCheckPoint**
  - 特定コアの特定番号のチェックポイントの通過を待機する
    - 同期対象の処理単位は処理後にチェックポイントを通過しなければならないため、処理後に実行状態でなくなる場合には利用できない
- **AktspStateSync**
  - 対象のタスクが指定した状態になるまで待機する

同期種別

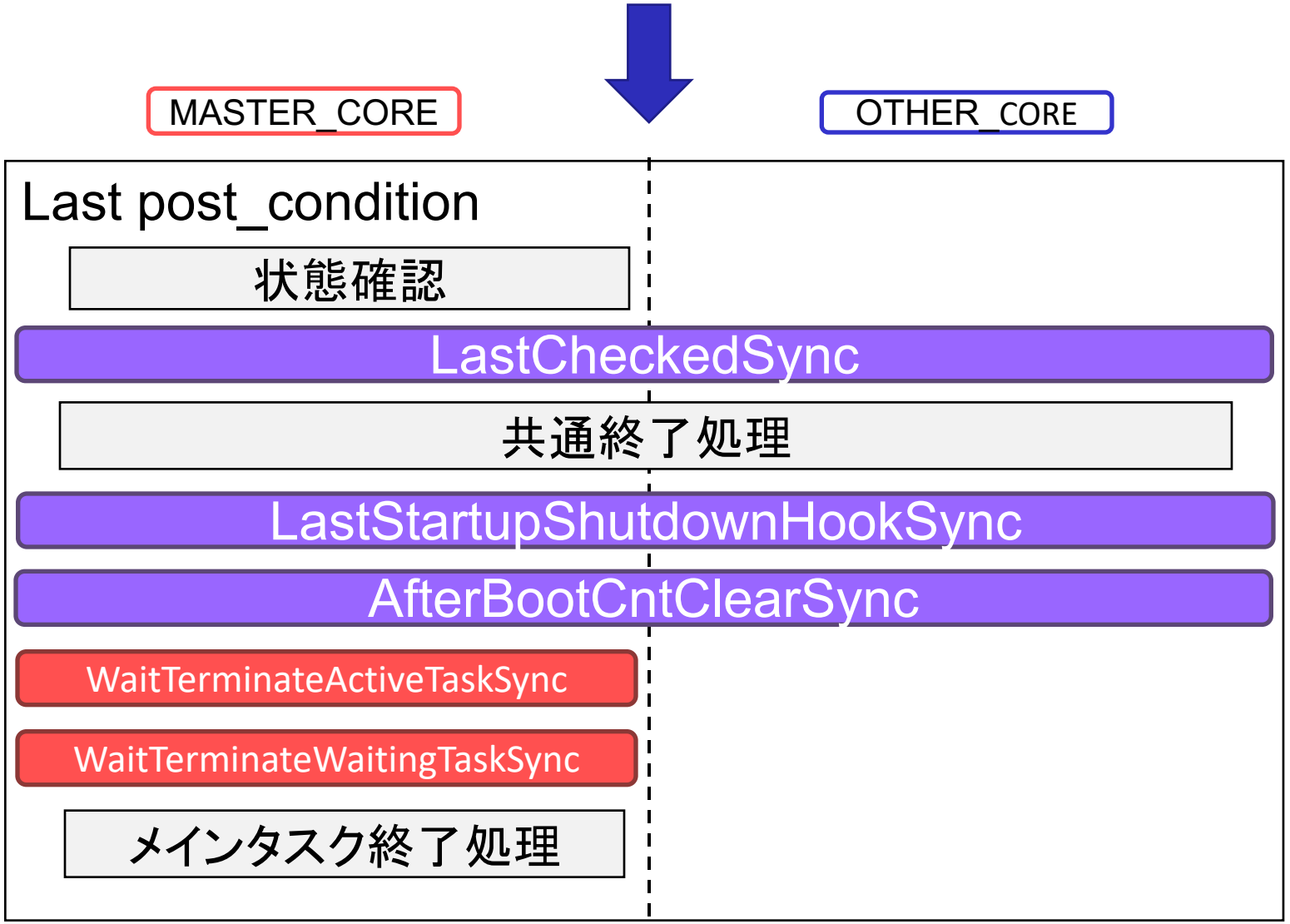
1. ActiveSync
2. StateSync
3. WaitActivateTaskSync
4. LockSync
5. ConditionSync
6. DoStartSync
7. DoFinishSync
8. LastCheckedSync
9. LastStartupShutdownHookSync
10. AfterBootCntClearSync
11. WaitTerminateActiveTaskSync
12. WaitTerminateWaitingTaskSync

同期メカニズム	同期種別
AktspBarrierSync	ConditionSync, WaitActivateTaskSync, LastCheckedSync, LastStartupShutdownHookSync, AfterBootCntClearSync
AktspMcCheckPoint	ActiveSync, LockSync, DoStartSync
AktspStateSync	StateSync, DoFinishSync, WaitTerminateActiveTaskSync, WaitTerminateWaitingTaskSync

処理の流れ①

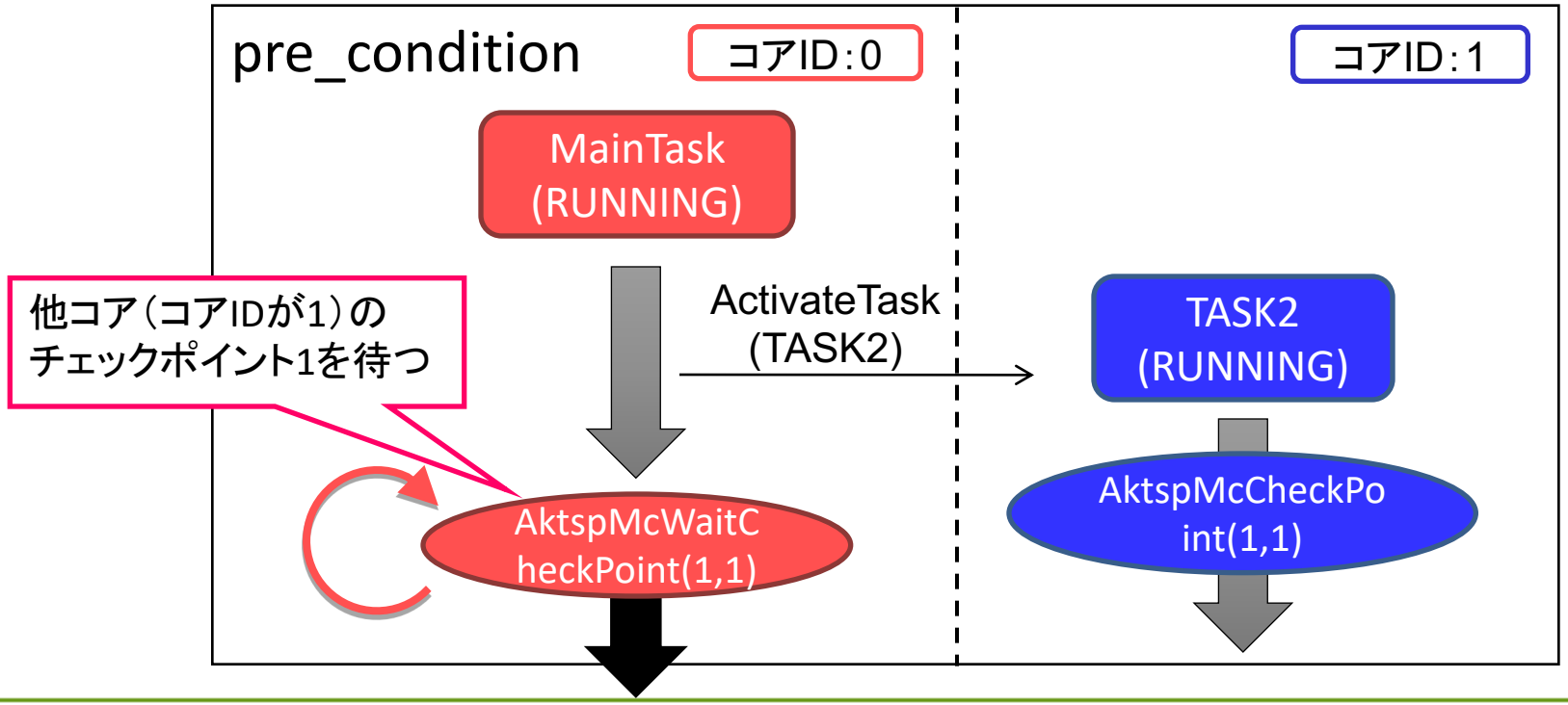

MASTER_CORE
OTHER_CORE
pre_condition
ActiveSync, StateSync, LockSync
WaitActivateTaskSync
状態確認
ConditionSync
do
DoStartSync
DoStartSync
DoFinishSync
ConditionSync
post_condition
状態確認
ConditionSync

処理の流れ②


MASTER_CORE
OTHER_CORE
Last post_condition
状態確認
LastCheckedSync
共通終了処理
LastStartupShutdownHookSync
AfterBootCntClearSync
WaitTerminateActiveTaskSync
WaitTerminateWaitingTaskSync
メインタスク終了処理

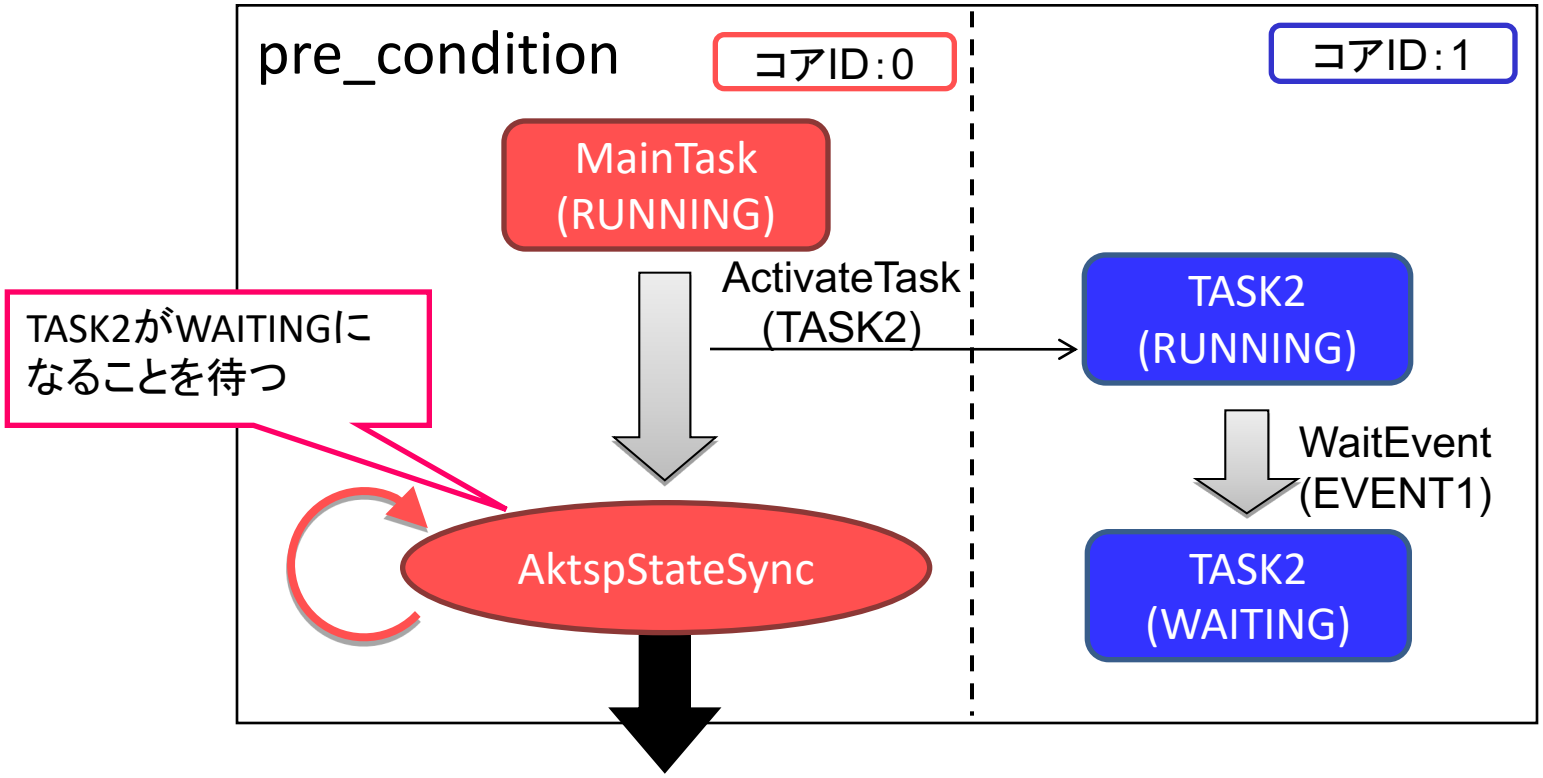
1. ActiveSync

- pre_conditionにて，他コアにおける処理単位の実行状態を待つ
  - 起動状態となる処理単位側でチェックポイントを発行し，起動元(発行元)で，そのチェックポイントを待つ
- AktspMcWaitCheckPointを用いる
- サンプルTESRYは，Task_0001.yaml



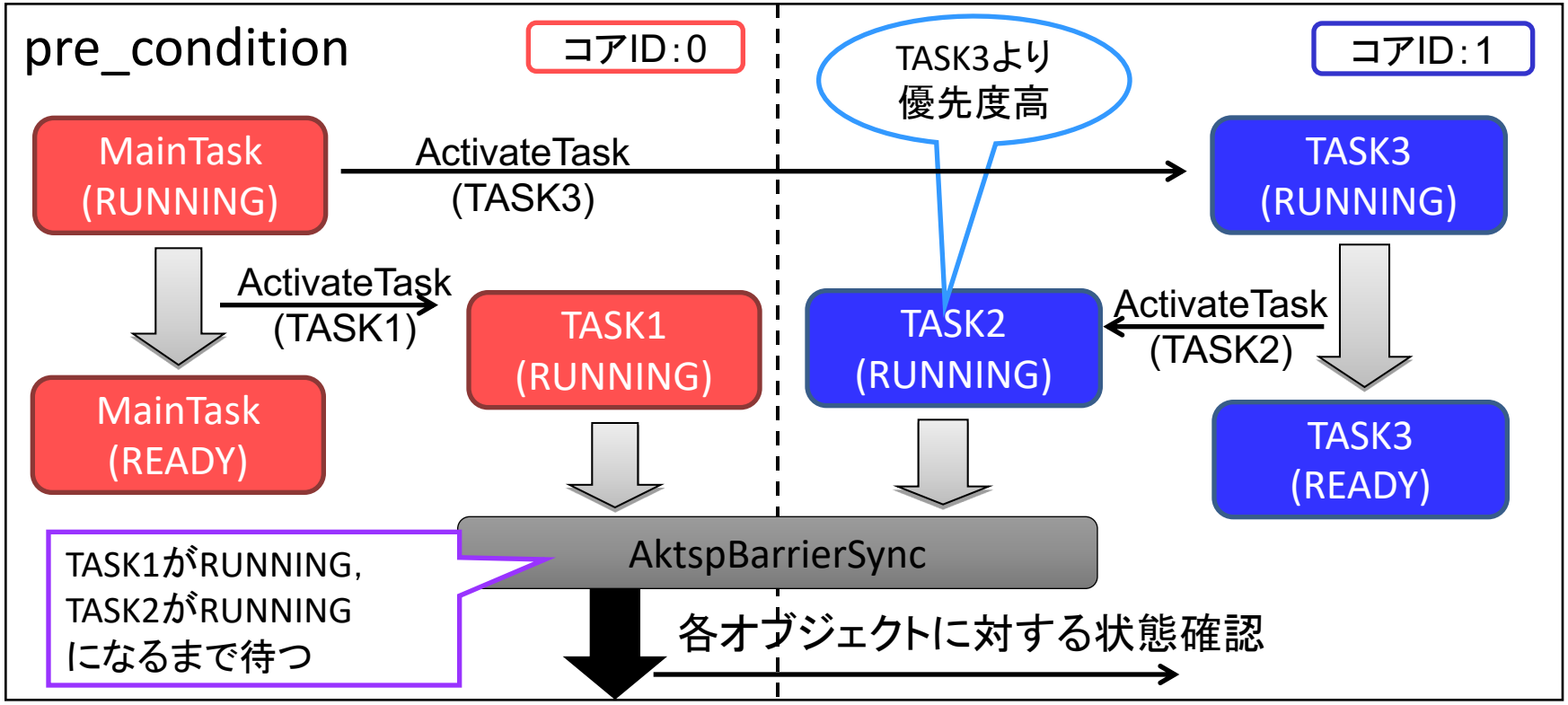
2. StateSync

- pre_conditionにて，他コアの対象タスクがイベント待ち状態になることを待つ
- AktspStateSyncを用いる
- サンプルTESRYは，Event_0004.yaml



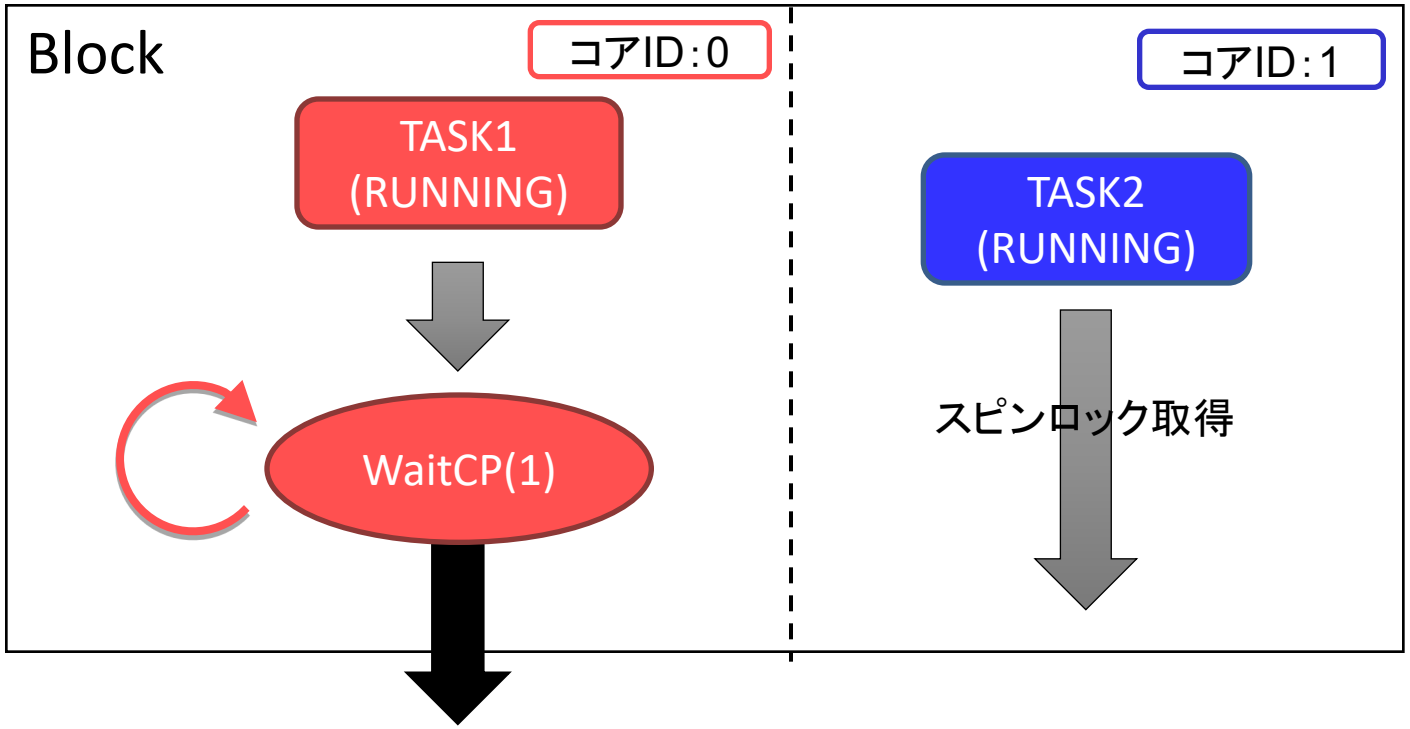
3. WaitActivateTaskSync

- pre_conditionにて，他コアのタスクの起動を待つ
  - 他コアの対象タスクが起動する前にマスタコアから状態確認することを防ぐ
- AktspBarrierSyncを用いる
- サンプルTSERYは，Task_0005.yaml



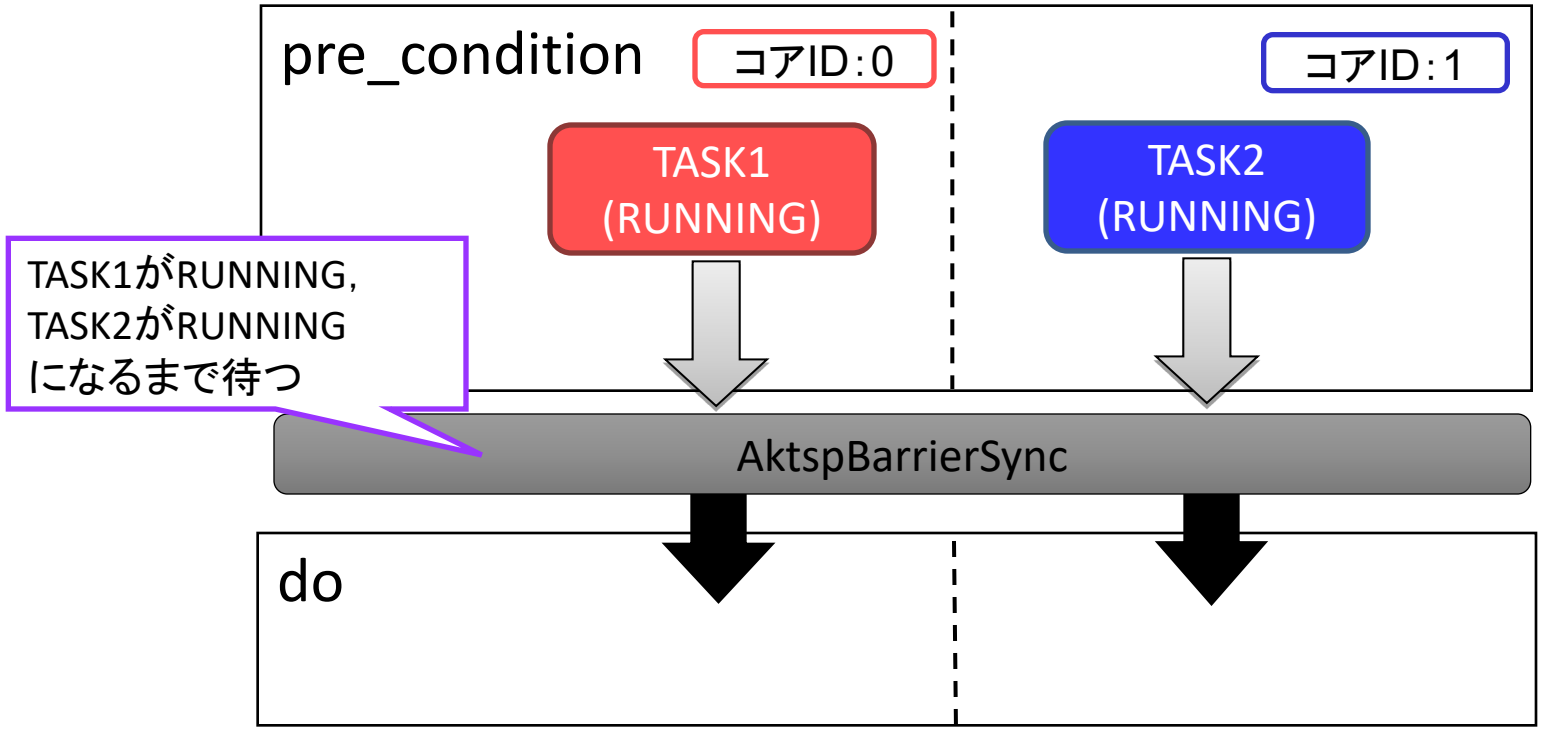
4. LockSync【未対応】

- 他コアにおける，処理単位のスピンロック取得を待つ
- ＊＊＊を用いる



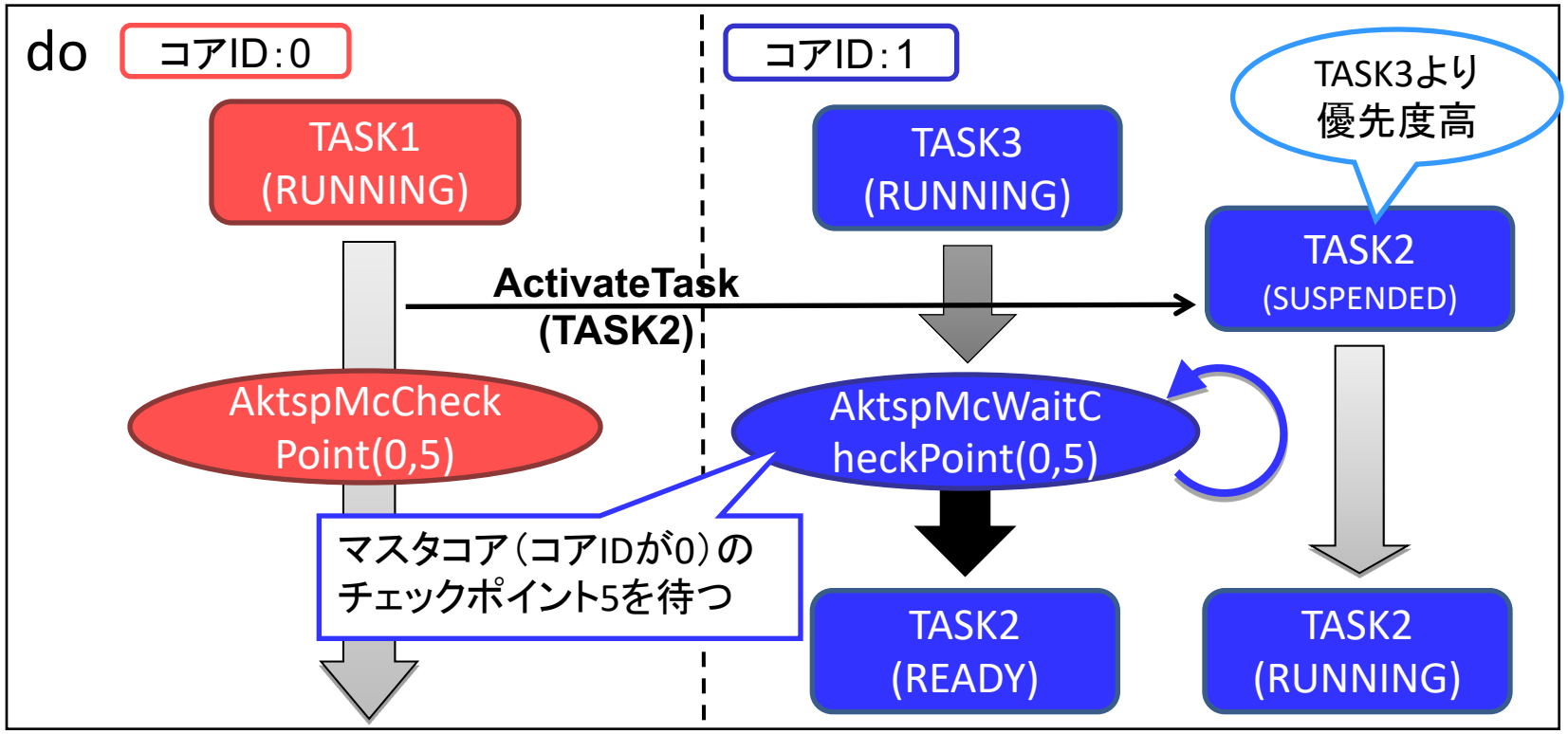
5. ConditionSync

- 各コンディション終了時に，複数のコアに実行状態の処理単位が存在する場合，各コアの実行状態の処理単位を同期する
- AktspBarrierSyncを用いる
- サンプルTESRYは，Task_0001.yaml（他全てのサンプルTESRY）



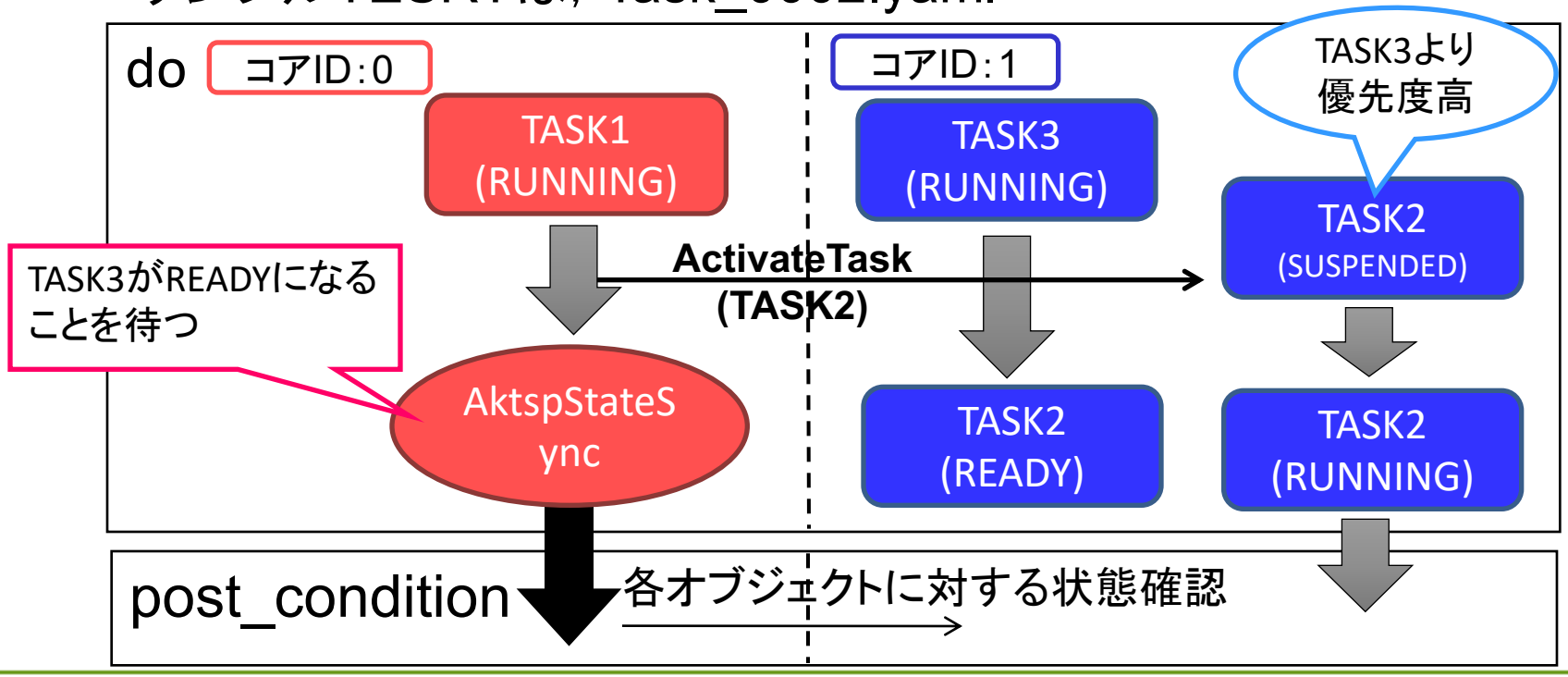
6. DoStartSync

- 他コアに対するシステムサービス発行により，他コアで実行状態であった処理単位が実行状態でなくなる場合，doのAPI発行終了まで実行状態のままで待つ
- AktspMcWaitCheckPointを用いる
- サンプルTESRYは，Task_0002.yaml



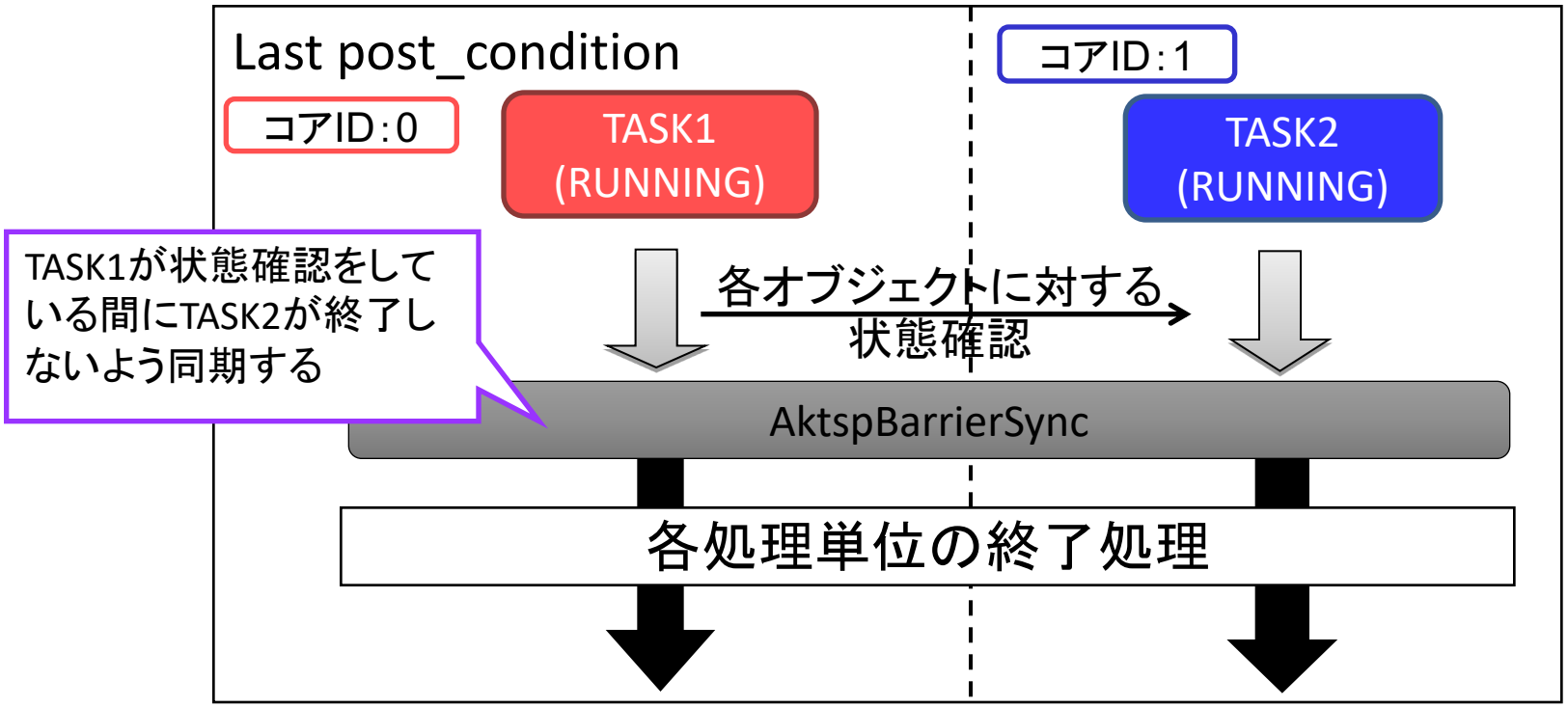
7. DoFinishSync

- 他コアにおいて，実行状態のタスクがdoの処理単位の結果として実行状態でなくなった場合，そのタスクが現在の状態になるまで待機する
  - タスクが実行状態でない状態になる前に状態確認を実行することを防ぐ
- AktspStateSyncを用いる
- サンプルTESRYは，Task_0002.yaml



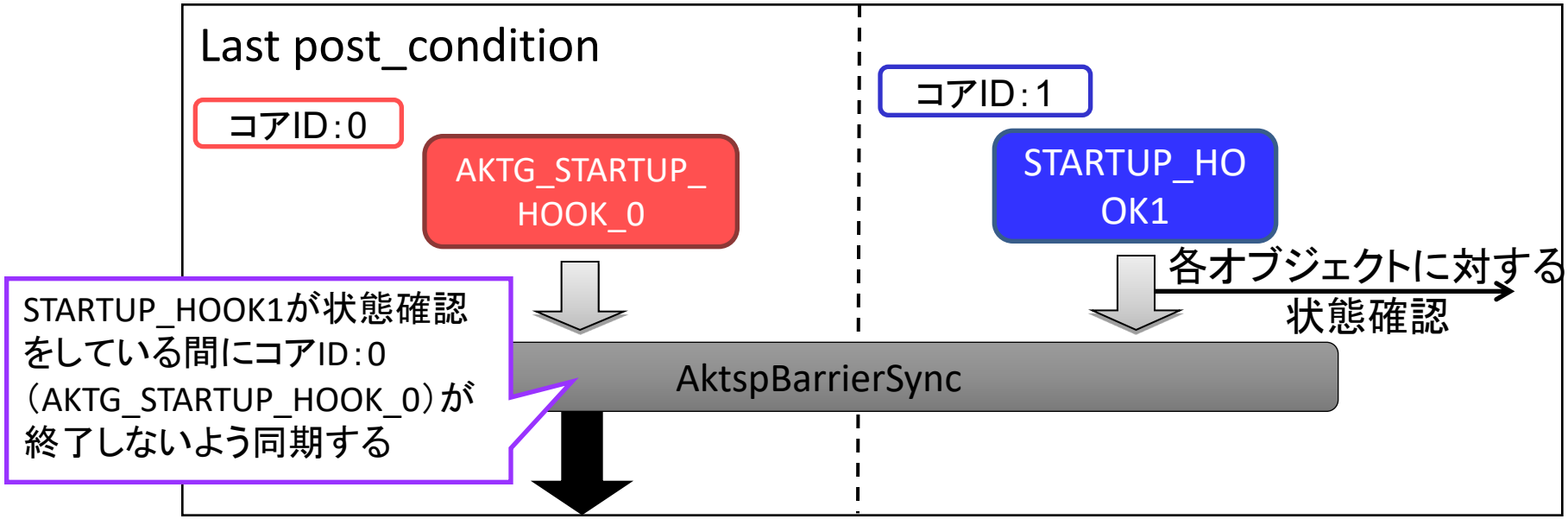
8. LastCheckedSync

- 最後のpost_conditionにおける状態確認が終了するのを待つ
  - 状態確認中に他コアの実行中の処理単位が終わらないよう同期する
- AktspBarrierSyncを用いる
- サンプルTESRYは，Task_0001.yaml（他全てのサンプルTESRY）



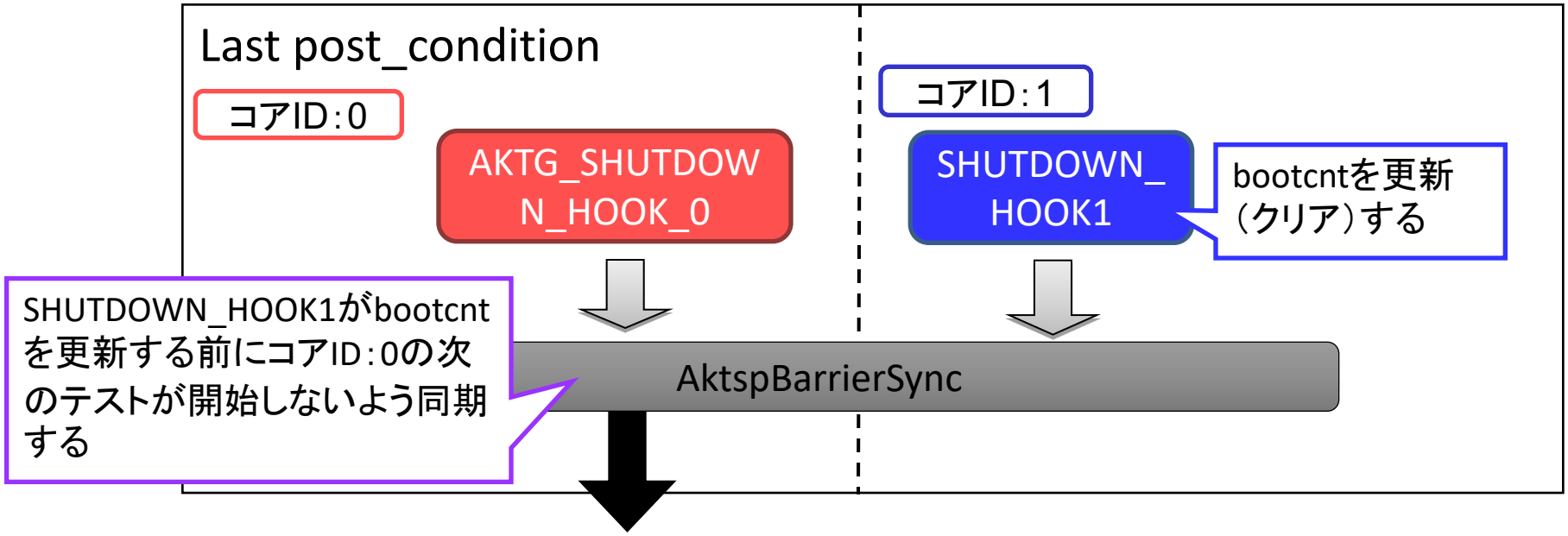
9. LastStartupShutdownHookSync

- マージ可能なStartup/ShutdownHookで終わるテストケースの最後のpost_conditionにて，Startup/ShutdownHookの処理が終了するのを待つ
  - マージした場合，各コアのStartup/ShutdownHook（補完した空の関数）が先行して次のテストへ進まないための同期
- AktspBarrierSyncを用いる
- サンプルTESRYは，C1ISR_0007.yaml



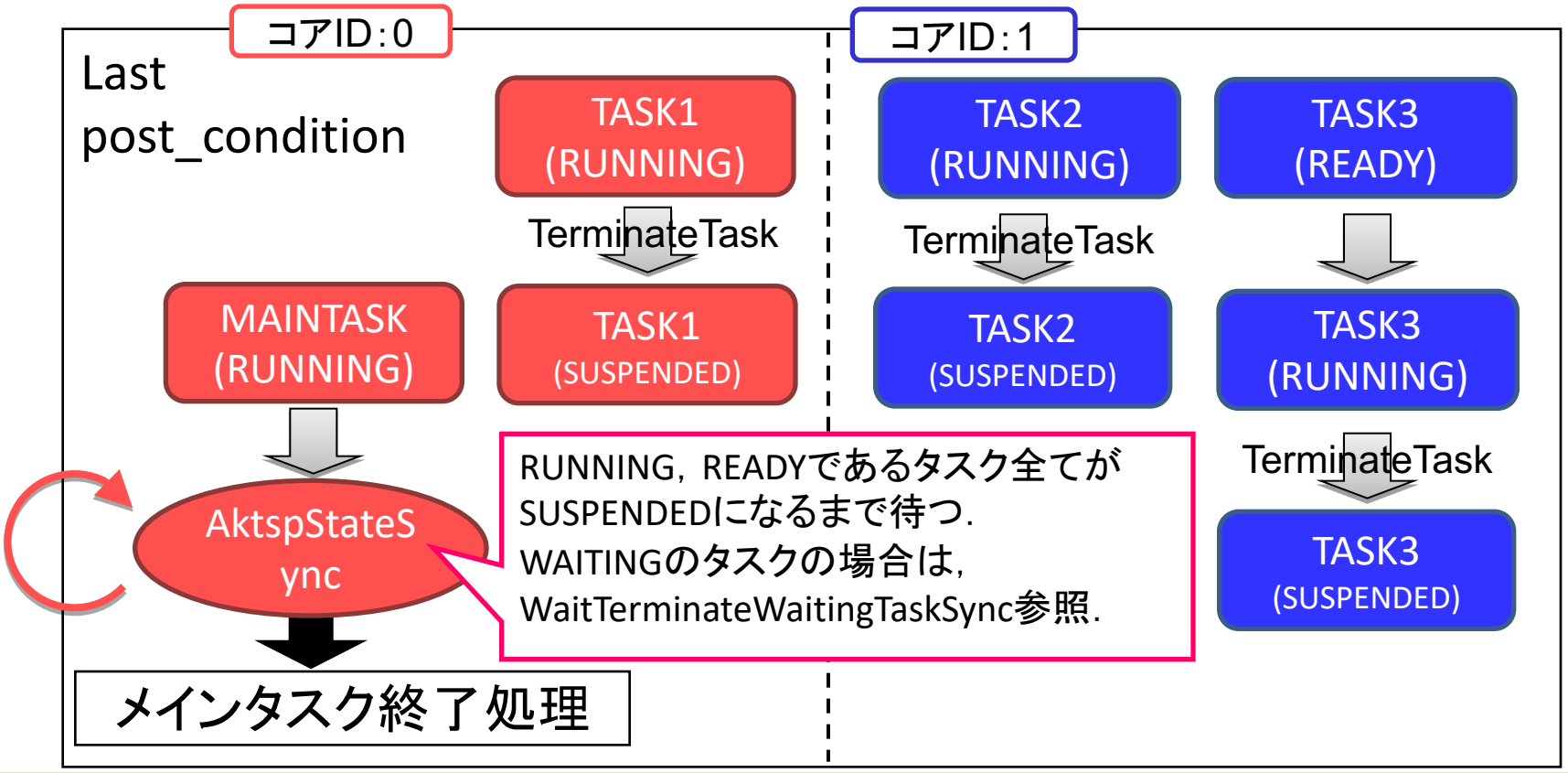
10. AfterBootCntClearSync

- マージ可能なStartup/ShutdownHookで終わるテストケースの最後のpost_conditionにて，bootcntの更新を待つ
  - 自コアのStartup/ShutdownHookでbootcntを更新する前に，他コアで同じbootcntを使用する次のテストが先行して進まないための同期
- AktspBarrierSyncを用いる
- サンプルTESRYは，C1ISR_0004.yaml



11. WaitTerminateActiveTaskSync

- 最後のpost_conditionにて，各コアのRUNNINGとREADYであるタスクがSUSPENDEDとなるまで，メインタスクから待つ
- AktspStateSyncを用いる
- サンプルTESRYは，Task_0005.yaml（他全てのサンプルTESRY）



12. WaitTerminateWaitingTaskSync

- 最後のpost_conditionにて，各コアのWAITINGであるタスクがSUSPENDEDとなるまで，メインタスクから待つ
- AktspStateSyncを用いる
- サンプルTESRYは，Task_0014.yaml

